Designing a RISCV CPU

Maxim Faber

Terminology

A Central Processing Unit (CPU)

- Executes the programs that a programmer loads onto it.
- A CPU can run any valid program provided it is compatible with the CPU's instruction set. It does not require an operating system.*
- CPUs are built using digital logic.

ISA (Instruction Set Architecture)

- Spec defining the semantics, behavior, and assembler of a CPUs operations.
- Examples
  - RISCV
  - x86
  - x86_64
  - ARMv8

Micro Architecture

- How the hardware designer chooses to impliment the ISA.

Micro Architecture

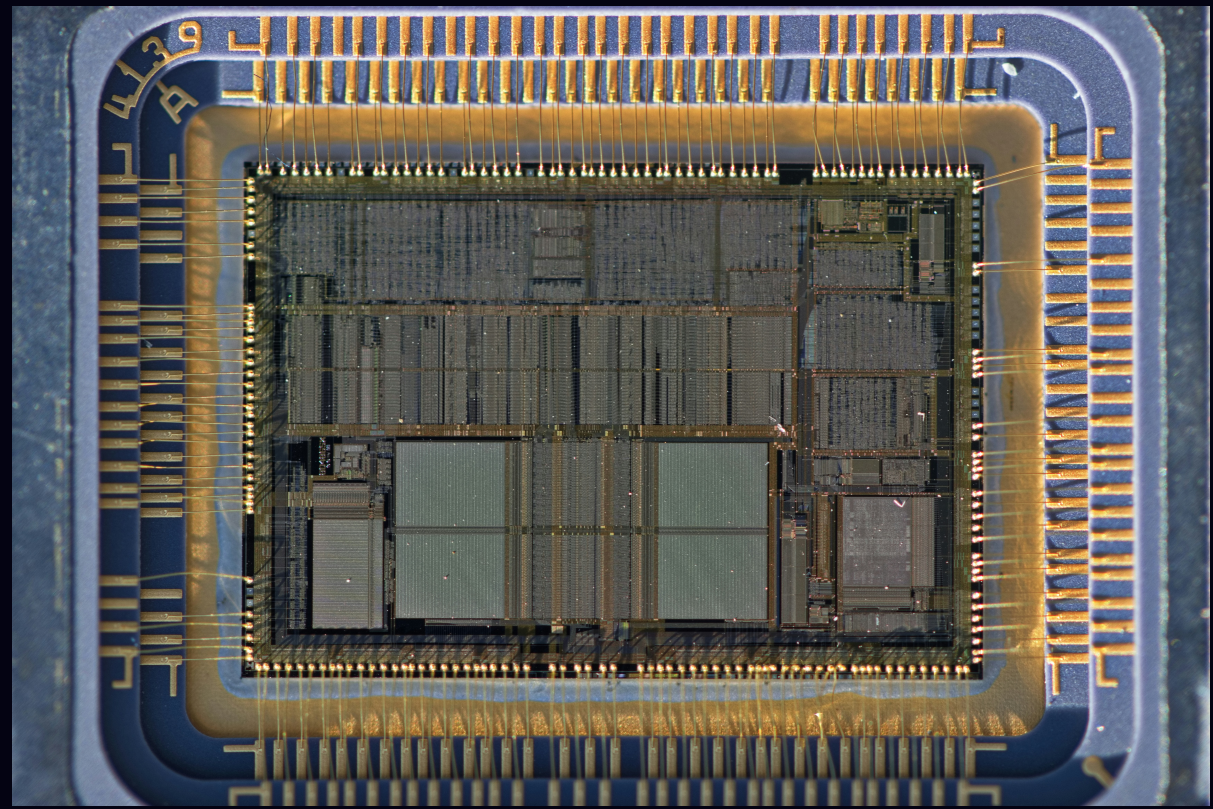
- How the hardware designer chooses to impliment the ISA.

State / Statefull

- Information about a system/process
- Example: Memory

Pure / Combinational

- A pure (side effect free) function of some inputs (like math functions)
- Example: Adder



License MIT

What's Inside ?

Memory

- Statefull elements
- Types
  - SRAM/SSRAM
  - DRAM/SDRAM
  - DDR, SDR
  - AND MANY MORE
- Operates on a clock
- You can read and write to it

Registers

- Type of memory
- VERY fast
- Stores a single* word (usually 64 bits)
- Grouped in register files
- Operates on a clock

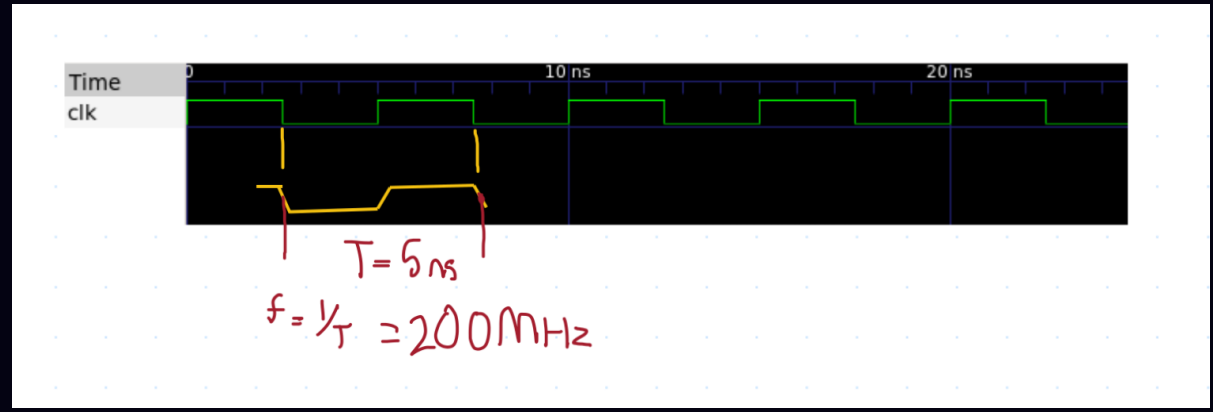
Combinational Logic

ALUs

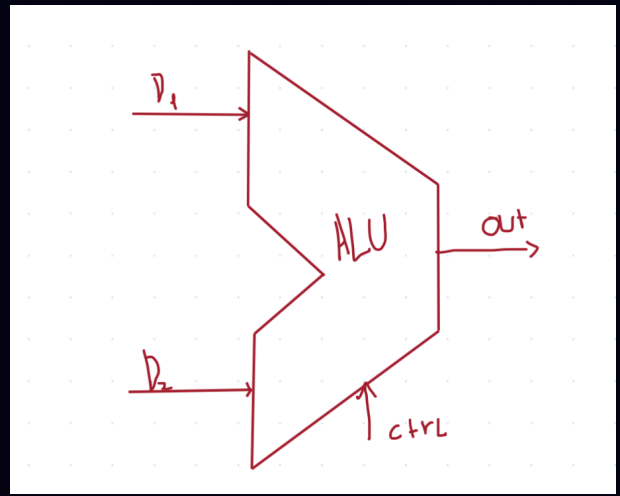
- Does the math outlined by the ISA and instructed by the program
- ADD, SUB, SR, SL...
- Must be told what operation to execute

Control

- What interprets the decoded instruction (next slide) and tells the components how to behave
- Sends control lines out to the alu (and others)



clk



License MIT

What's Inside ?

Memory

- Statefull elements
- Types
  - SRAM/SSRAM
  - DRAM/SDRAM
  - DDR, SDR
  - AND MANY MORE
- Operates on a clock
- You can read and write to it

Registers

- Type of memory
- VERY fast
- Stores a single* word (usually 64 bits)
- Grouped in register files
- Operates on a clock

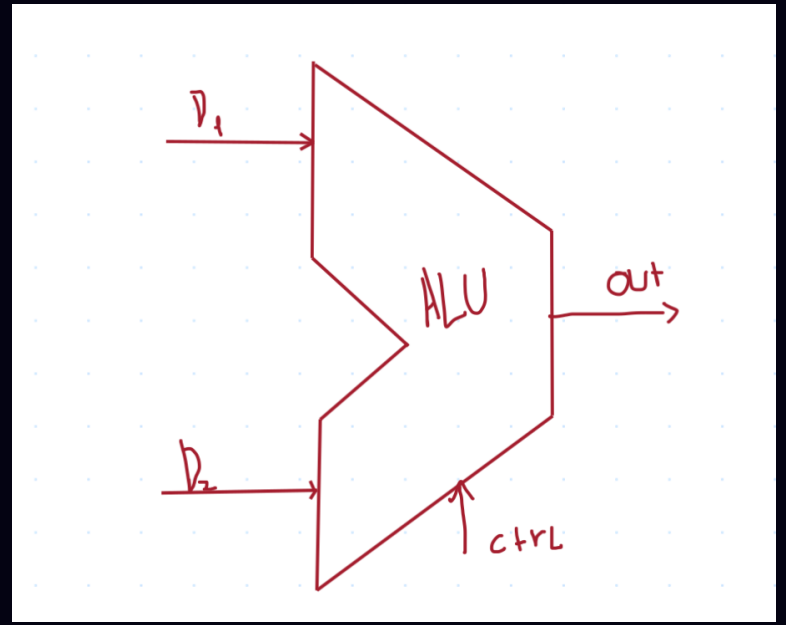
Combinational Logic

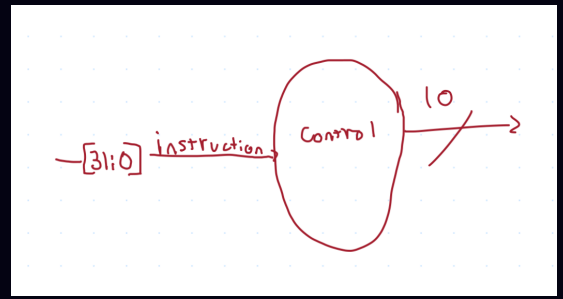
ALUs

- Does the math outlined by the ISA and instructed by the program
- ADD, SUB, SR, SL...
- Must be told what operation to execute

Control

- What interprets the decoded instruction (next slide) and tells the components how to behave
- Sends control lines out to the alu (and others)



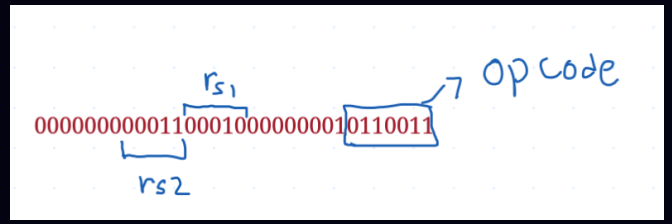


License MIT

Putting it All together

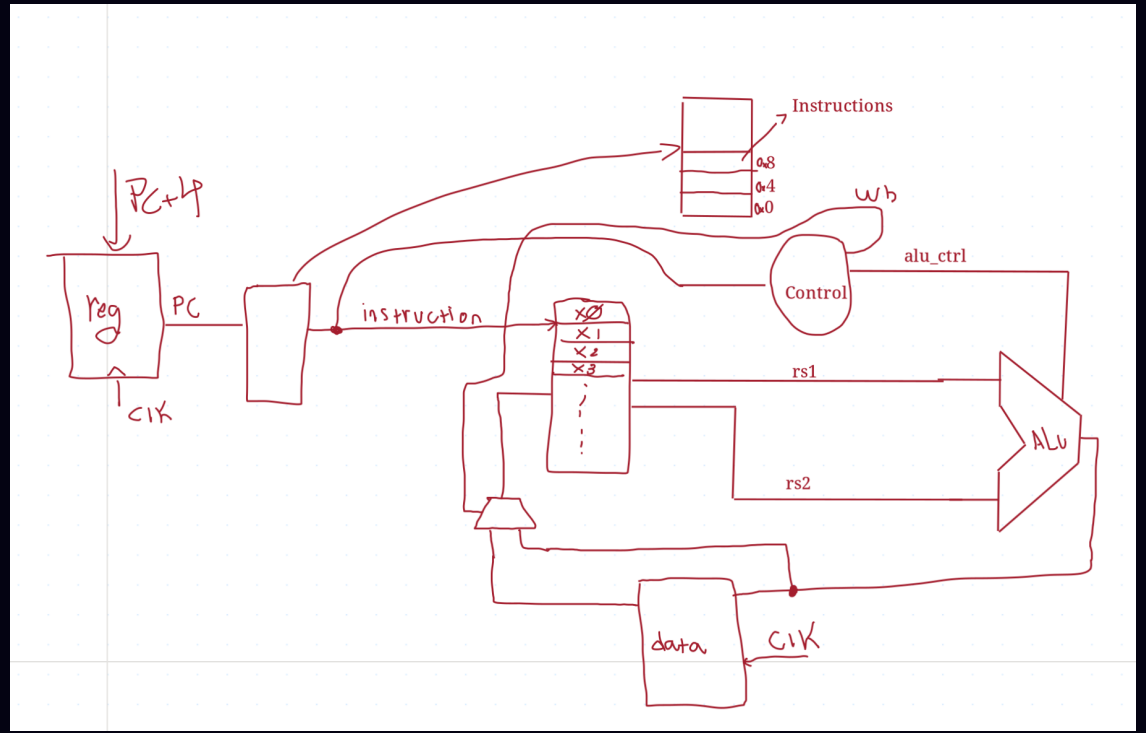
Assembly:

- add x1, x2, x3
- [00000000001100010000000010110011] (in binary)



Restrictions

- This all must happen in a single clock cycle !!!



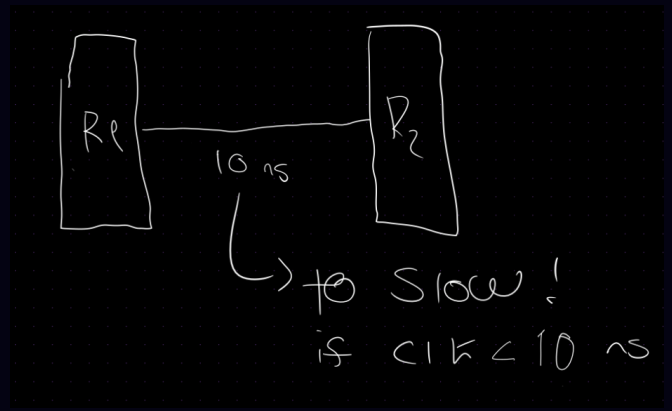
License MIT

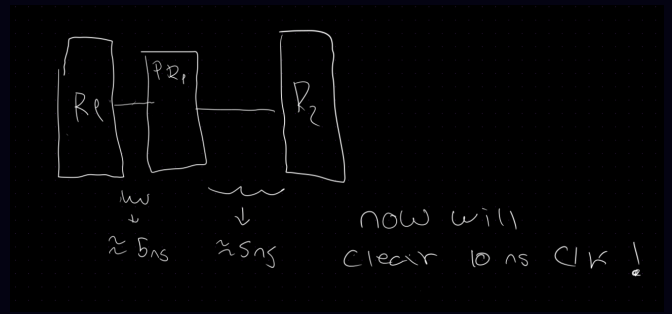
Restrictions pt 2

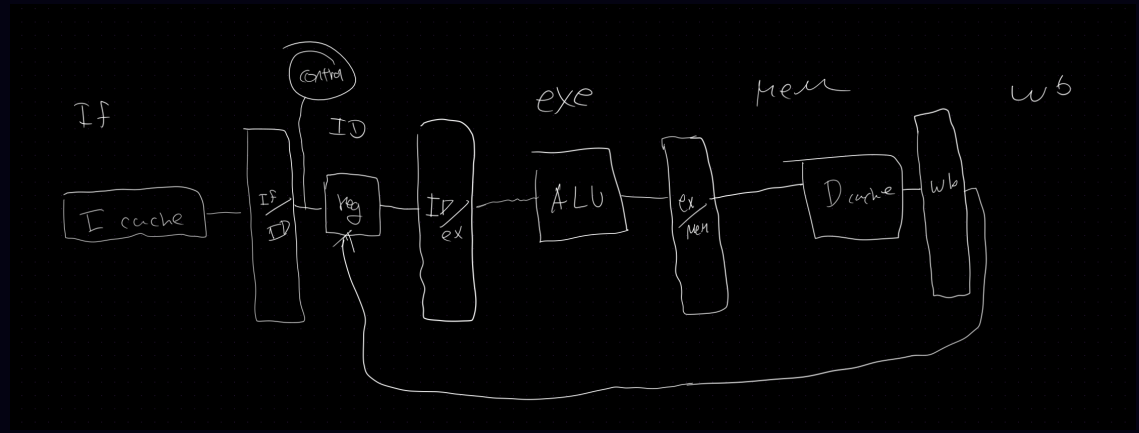
- Combinational path is really long
- High prop delay

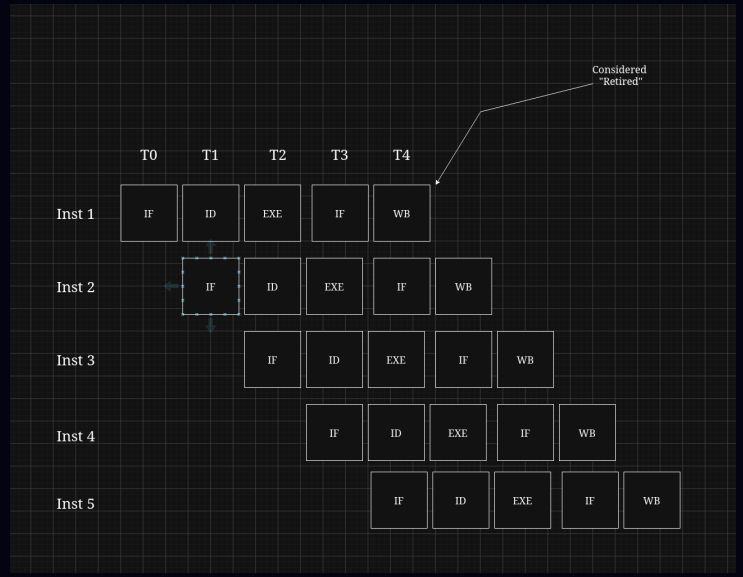
Pipelining

- Breaks combinational paths with statefull elements









License MIT

🟦 Tools Needed

- git
- Sim: verilator, icarus verilog
- Verilog, SystemVerilog
- testing: cocotb
- compiler: riscv64-unknown-elf-gcc (linux/mac)

Resources:

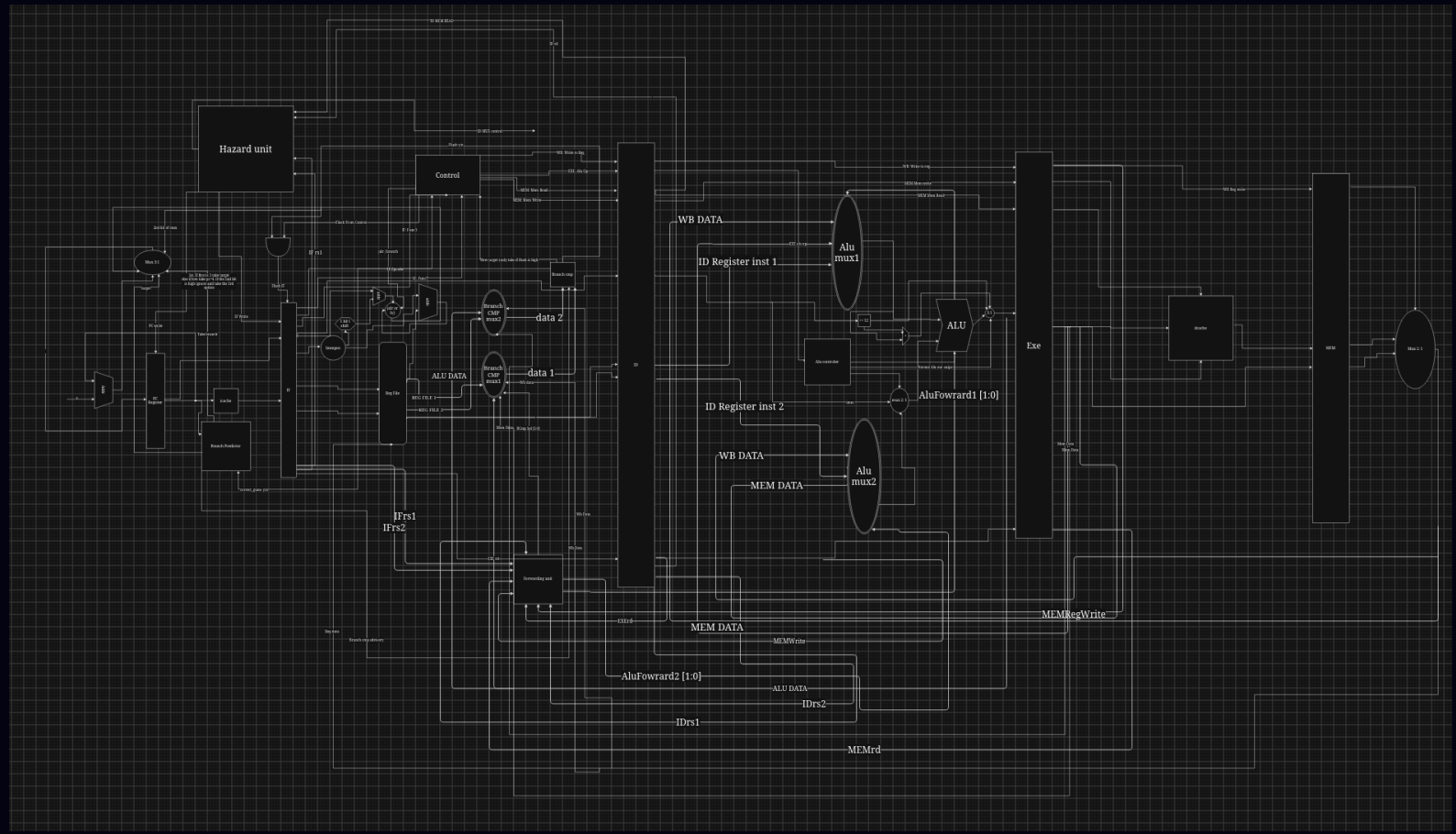
- libary: https://learning.oreilly.com/

Books:

- Computer Organization and Desing (ch 2, 4)
- Computer Architecture A quantitative approach
- ARM Fundamentals of SoC

Websites:

- https://scalable.uni-jena.de/opt/ce/index.html
- https://docs.riscv.org/reference/isa/v20260120/index.html
- https://luplab.gitlab.io/rvcodecjs/


Hazard unit
Control
WB DATA
ID Register inst 1
Alu mux1
ALU
Exe
ID Register inst 2
AluFowrard1 [1:0]
WB DATA
MEM DATA
Alu mux2
data 2
data 1
ALU DATA
IFrs1
IFrs2
MEMRegWrite
MEM DATA
MEMWrite
AluFowrard2 [1:0]
ALU DATA
IDrs2
IDrs1
MEMrd

License MIT

The end

maxim.faber@protonmail.com
Open a issue or email with topic requests / questions

License MIT